# Generating Chest X-Rays with a Convolutional VAE

**Yun-Tong Tsai**

## Abstract

A PyTorch variational autoencoder (VAE) generates $64 \times 64$ chest radiographs. After 30 epochs, Inception Score is 1.697 and torchvision-feature FID is 210.96. Samples capture coarse chest structure but lack fine detail.

## 1 Data and model

Normal and pneumonia images from Kaggle [1] are pooled. SHA-256 deduplication removes 32 files, retaining 618 official test images. Training excludes 453 images whose filename-derived patient IDs occur in test; 10% of remaining shuffled groups form validation (seed 42): 4269 training and 468 validation images. The 16 official validation images are unused. Preprocessing uses grayscale conversion, PIL bilinear resize to $64 \times 64$, and scaling to $[0, 1]$, without augmentation; aspect ratios can change.

The encoder uses four $4 \times 4$ convolutions (stride 2, padding 1; channels $1, 32, 64, 128, 256$; SiLU), then linear heads for 32-dimensional $\mu$ and $\log \sigma^2$. Reparameterization $z = \mu + \sigma \odot \epsilon, \epsilon \sim \mathcal{N}(0, I)$, enables backpropagation [2]. A linear projection and four transposed convolutions reverse these dimensions; sigmoid outputs intensities. The model has 1,775,361 parameters, without batch normalization.

## 2 Training protocol and learning curves

BCE sums over 4,096 pixels; KL sums over latent dimensions. Minibatch means minimize:

$$\mathcal{L}_\beta = \mathbb{E}_{q_\phi(z|x)}[-\log p_\theta(x|z)] + \frac{\beta}{2} \sum_{j=1}^{32} (\mu_j^2 + \sigma_j^2 - 1 - \log \sigma_j^2). \tag{1}$$

BCE is a soft-target surrogate for continuous intensities. Settings: Adam learning rate $10^{-3}$, moments $(0.9, 0.999)$, batch size 64, zero weight decay, gradient-norm limit 10, 30 epochs, seed 42, and warm-up $\beta = \min(1, e/5)$ at epoch $e$. Validation uses fixed posterior noise and $\beta = 1$; its minimum selects epoch 30 (2391.81 nats/image).

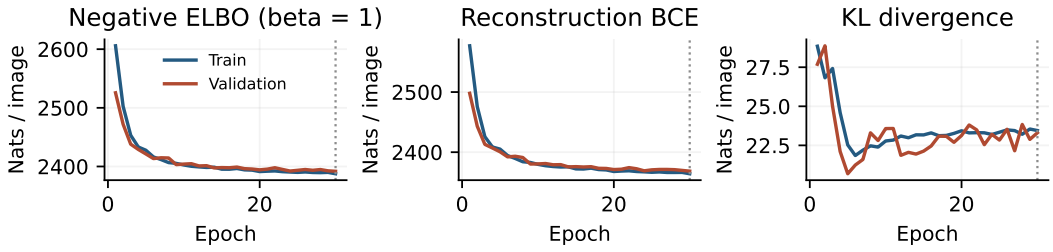



Figure 1: Losses per image; the left panel uses $\beta = 1$ even during warm-up. Training values average across updates; dotted lines mark the selected checkpoint.

## 3 Generation and quantitative evaluation

The selected decoder generates 618 prior samples from $z \sim \mathcal{N}(0, I)$ (seed 123), compared with all 618 test images using Ignite's FID/IS evaluator [3]. Both sets receive 8-bit conversion, RGB replication, PIL bilinear resize to $299 \times 299$, and ImageNet normalization (an addition to the tutorial). Frozen torchvision Inception-v3 (`IMAGENET1K_V1`) supplies 2,048 pooled features for the primary FID and 1,000 softmax probabilities for IS and the Ignite-default FID representation. Neither FID is comparable to TensorFlow-weight `pytorch-fid` scores. Generated images are sigmoid decoder means, with no visual selection.

Table 1: Test results at epoch 30. Losses: nats/image. Split IS reports mean and standard deviation over ten subsets, not independently trained models.

| Metric | Value | Metric | Value |
|---|---|---|---|
| FID (2,048 pool features)↓ | 210.96 | Test negative ELBO↓ | 2391.06 |
| FID (1,000 probabilities)↓ | 0.28299 | Test reconstruction BCE↓ | 2367.58 |
| IS (all samples)↑ | 1.697 | Test KL | 23.47 |
| IS (10 splits)↑ | $1.677 \pm 0.080$ | Real / generated count | 618 / 618 |

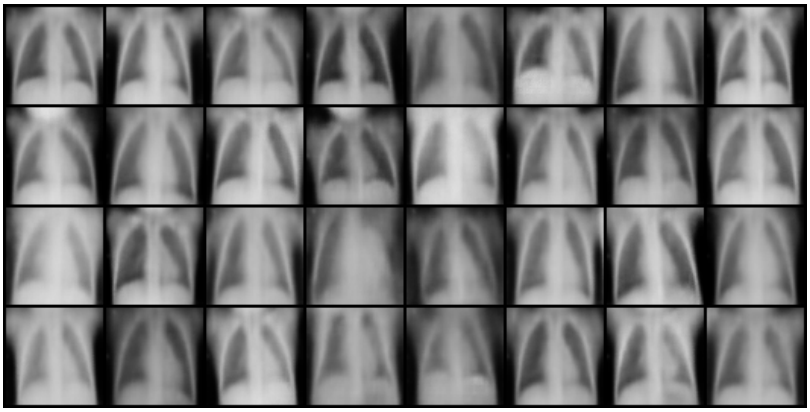


Figure 2: The first 32 generated images from evaluation seed 123, in order. All panels use the same $[0, 1]$ display range.

## 4 Discussion and limitations

Validation negative ELBO decreases from 2,525.63 at epoch 1 to 2,391.81 at epoch 30, while the final training value is 2,387.57. The small gap provides no clear evidence of overfitting in this run. Generated samples capture broad lung fields and a central bright region, with variation in width and intensity, but fine structures are blurred and some boundaries are distorted. The nonzero test KL indicates that the posterior differs from the prior; it alone does not prove that every latent dimension is useful. Pixelwise reconstruction and the low spatial resolution likely contribute to the limited detail. Filename grouping cannot verify clinical identity or near-duplicates. With only 618 test images, the 2,048-feature covariance is rank deficient and FID is affected by finite-sample bias. ImageNet-based IS cannot establish pathology coverage or anatomical correctness. Only one training and sampling seed are evaluated. Higher resolution, repeated runs, and radiograph-specific evaluation are useful next steps.

**Code:** `https://github.com/tsaiton123/ECE285-DeepGenerativeModel-HW1`

## References


[1] P. Mooney. *Chest X-Ray Images (Pneumonia)*. Kaggle.
[2] D. P. Kingma and M. Welling. Auto-Encoding Variational Bayes. ICLR, 2014. arXiv:1312.6114.
[3] PyTorch-Ignite. GAN evaluation using FID and IS.

# Source Code

PyTorch VAE implementation, data preparation, training, Ignite FID/IS evaluation, image generation, and tests. File paths and original line numbers are shown below. Python 3.12 was used.

## requirements.txt

```
torch==2.14.1
torchvision==0.29.1
pytorch-ignite==0.5.5
numpy==2.5.4
scipy==1.18.1
Pillow==13.0.0
matplotlib==3.11.2
pypdf==6.21.0
pytest==9.1.1
```

## chest_vae/__init__.py

```
"""Reproducible chest X-ray variational autoencoder experiment."""
```

## chest_vae/model.py

```
import torch
from torch import nn
from torch.nn import functional as F


class VAE(nn.Module):
    """64 x 64 grayscale VAE; decoder returns Bernoulli logits."""

    def __init__(self, latent_dim=32, base_channels=32):
        super().__init__()
        self.latent_dim = latent_dim
        b = base_channels
        layers = []
        for cin, cout in zip([1, b, 2*b, 4*b], [b, 2*b, 4*b, 8*b]):
            layers += [nn.Conv2d(cin, cout, 4, 2, 1), nn.SiLU()]
        self.encoder = nn.Sequential(*layers, nn.Flatten())
        self.mu = nn.Linear(8*b*4*4, latent_dim)
        self.logvar = nn.Linear(8*b*4*4, latent_dim)
        self.project = nn.Linear(latent_dim, 8*b*4*4)
        self.decoder = nn.Sequential(
            nn.Unflatten(1, (8*b, 4, 4)),
            nn.ConvTranspose2d(8*b, 4*b, 4, 2, 1), nn.SiLU(),
            nn.ConvTranspose2d(4*b, 2*b, 4, 2, 1), nn.SiLU(),
            nn.ConvTranspose2d(2*b, b, 4, 2, 1), nn.SiLU(),
            nn.ConvTranspose2d(b, 1, 4, 2, 1),
        )

    def encode(self, x):
        h = self.encoder(x)
        return self.mu(h), self.logvar(h)

    def decode(self, z):
        return self.decoder(self.project(z))

    def forward(self, x, eps=None):
        mu, logvar = self.encode(x)
        if eps is None:
            eps = torch.randn_like(mu)
        z = mu + torch.exp(0.5 * logvar) * eps
        return self.decode(z), mu, logvar


def loss_terms(logits, x, mu, logvar, beta=1.0):
```

```
    """Sum over pixels/latent dimensions, then mean over examples."""
    recon = F.binary_cross_entropy_with_logits(logits, x, reduction="none").flatten(1).sum(1)
    kl = 0.5 * (mu.square() + logvar.exp() - 1 - logvar).sum(1)
    return (recon + beta*kl).mean(), recon.mean(), kl.mean()
```

## chest_vae/data.py

```
"""Data preparation with an explicit, reproducible split manifest."""
import argparse
from collections import Counter
import hashlib
import json
from pathlib import Path
import random
import re
import zipfile

import numpy as np
from PIL import Image
import torch


def patient_key(name):
    # Patient identifiers inferred from the published filenames, not clinical metadata.
    m = re.match(r"(person\d+)_", name)
    if m:
        return m.group(1)
    m = re.match(r"(NORMAL2-IM-\d+|IM-\d+)-", name)
    if m:
        return m.group(1)
    raise ValueError(f"Unrecognized patient filename: {name}")


def prepare(archive, output, seed=42, val_fraction=0.1):
    output = Path(output)
    output.mkdir(parents=True, exist_ok=True)
    with zipfile.ZipFile(archive) as z:
        # The Kaggle archive repeats the dataset under nested directories.
        candidates = [n for n in z.namelist() if re.search(r"/train/NORMAL/[^/]+\.jpeg$", n)]
        if not candidates:
            raise ValueError("Expected Kaggle chest_xray/train/NORMAL/*.jpeg")
        prefix = min((n.split("train/NORMAL/")[0] for n in candidates), key=lambda s: (len(s), s))
        entries = []
        seen_hashes = {}
        duplicates = []
        for split in ["test", "train", "val"]:
            names = sorted(n for n in z.namelist() if n.startswith(prefix + split + "/")
                           and n.lower().endswith((".jpeg", ".jpg", ".png")) and "/__" not in n)
            for name in names:
                relative = name[len(prefix):]
                if len(Path(relative).parts) != 3:
                    continue
                raw = z.read(name)
                digest = hashlib.sha256(raw).hexdigest()
                if digest in seen_hashes:
                    duplicates.append({"removed": relative, "kept": seen_hashes[digest]})
                    continue
                seen_hashes[digest] = relative
                entries.append({"path": relative, "official_split": split,
                                "label": Path(relative).parts[1], "patient": patient_key(Path(name).name),
                                "sha256": digest, "archive_path": name})
        test_groups = {e["patient"] for e in entries if e["official_split"] == "test"}
        train_pool = [e for e in entries if e["official_split"] == "train" and e["patient"] not in test_groups]
        groups = sorted({e["patient"] for e in train_pool})
        random.Random(seed).shuffle(groups)
        val_groups = set(groups[:max(1, round(len(groups)*val_fraction))])
        arrays = {k: [] for k in ["train", "val", "test"]}
        for e in entries:
```

```
            source = e["official_split"]
            e["split"] = ("test" if source == "test" else
                          "unused" if source == "val" or e["patient"] in test_groups else
                          "val" if e["patient"] in val_groups else "train")
            if e["split"] != "unused":
                import io
                with Image.open(io.BytesIO(z.read(e["archive_path"]))) as im:
                    im = im.convert("L").resize((64, 64), Image.Resampling.BILINEAR)
                    arrays[e["split"]].append(np.asarray(im).copy())
        for k, images in arrays.items():
            np.save(output / f"{k}.npy", np.stack(images))
    summary = {k: dict(Counter(e["label"] for e in entries if e["split"] == k))
               for k in ["train", "val", "test", "unused"]}
    manifest = {"seed": seed, "val_group_fraction": val_fraction, "archive_prefix": prefix,
                "preprocessing": "PIL grayscale; direct bilinear resize to 64x64; uint8/255; no augmentation",
                "counts": summary, "duplicate_files_removed": duplicates, "entries": entries}
    (output / "manifest.json").write_text(json.dumps(manifest, indent=2))
    print(json.dumps(summary, indent=2), flush=True)


def load_split(data_dir, split):
    x = np.load(Path(data_dir) / f"{split}.npy")
    return torch.from_numpy(x).unsqueeze(1).float().div_(255)


if __name__ == "__main__":
    p = argparse.ArgumentParser()
    p.add_argument("--archive", default="data/chest-xray-pneumonia.zip")
    p.add_argument("--output", default="data/prepared")
    p.add_argument("--seed", type=int, default=42)
    args = p.parse_args()
    prepare(args.archive, args.output, args.seed)
```

## chest_vae/train.py

```
import argparse
import csv
import json
import platform
from pathlib import Path
import random
import time

import numpy as np
import torch
from torch.utils.data import DataLoader
from torchvision.utils import save_image

from .data import load_split
from .model import VAE, loss_terms


def device_for(name):
    if name != "auto":
        return torch.device(name)
    return torch.device("cuda" if torch.cuda.is_available() else "mps" if torch.backends.mps.is_available() else "
     cpu")


def main():
    p = argparse.ArgumentParser()
    p.add_argument("--data", default="data/prepared")
    p.add_argument("--output", default="runs/vae")
    p.add_argument("--epochs", type=int, default=30)
    p.add_argument("--batch-size", type=int, default=64)
    p.add_argument("--latent-dim", type=int, default=32)
    p.add_argument("--base-channels", type=int, default=32)
    p.add_argument("--lr", type=float, default=0.001)
```

```
    p.add_argument("--warmup", type=int, default=5)
    p.add_argument("--seed", type=int, default=42)
    p.add_argument("--device", default="auto")
    a = p.parse_args()
    random.seed(a.seed); np.random.seed(a.seed); torch.manual_seed(a.seed)
    torch.set_num_threads(6)
    device = device_for(a.device)
    out = Path(a.output); out.mkdir(parents=True, exist_ok=True)
    if (out / "history.csv").exists():
        raise FileExistsError("Choose a new --output to preserve existing experiment results")
    train, val = load_split(a.data, "train"), load_split(a.data, "val")
    loader = DataLoader(train, batch_size=a.batch_size, shuffle=True,
                        generator=torch.Generator().manual_seed(a.seed), num_workers=0)
    model = VAE(a.latent_dim, a.base_channels).to(device)
    optimizer = torch.optim.Adam(model.parameters(), lr=a.lr)
    config = vars(a) | {"device_used": str(device), "torch": str(torch.__version__),
                        "python": platform.python_version(), "platform": platform.platform(),
                        "parameters": sum(p.numel() for p in model.parameters()),
                        "n_train": len(train), "n_val": len(val)}
    (out / "config.json").write_text(json.dumps(config, indent=2))
    print(json.dumps(config), flush=True)
    fixed_z = torch.randn(32, a.latent_dim, generator=torch.Generator().manual_seed(2026)).to(device)
    best = float("inf"); started = time.perf_counter()
    fields = ["epoch", "beta", "train_objective", "train_bce", "train_kl", "train_elbo_loss",
              "val_elbo_loss", "val_bce", "val_kl", "seconds"]
    with (out / "history.csv").open("w") as f:
        writer = csv.DictWriter(f, fieldnames=fields); writer.writeheader()
        for epoch in range(1, a.epochs+1):
            t = time.perf_counter()
            beta = min(1.0, epoch / max(1, a.warmup))
            model.train(); sums = np.zeros(3)
            for x in loader:
                x = x.to(device)
                optimizer.zero_grad(set_to_none=True)
                logits, mu, logvar = model(x)
                loss, bce, kl = loss_terms(logits, x, mu, logvar, beta=beta)
                if not torch.isfinite(loss):
                    raise FloatingPointError("Non-finite training objective")
                loss.backward()
                torch.nn.utils.clip_grad_norm_(model.parameters(), 10.0)
                optimizer.step()
                sums += np.array([loss.item(), bce.item(), kl.item()])*len(x)
            sums /= len(train)
            model.eval(); vs = np.zeros(3)
            # Fixed Monte Carlo noise makes validation curves comparable across epochs.
            eps_rng = torch.Generator().manual_seed(1000+a.seed)
            with torch.no_grad():
                for x in val.split(a.batch_size):
                    x = x.to(device)
                    eps = torch.randn(len(x), a.latent_dim, generator=eps_rng).to(device)
                    logits, mu, logvar = model(x, eps)
                    values = loss_terms(logits, x, mu, logvar)
                    vs += np.array([v.item() for v in values])*len(x)
            vs /= len(val)
            row = dict(zip(fields, [epoch, beta, *sums, sums[1]+sums[2], *vs, time.perf_counter()-t]))
            writer.writerow(row); f.flush()
            print(json.dumps(row), flush=True)
            checkpoint = {"model": {k: v.detach().cpu() for k,v in model.state_dict().items()},
                          "config": config, "epoch": epoch, "val_elbo_loss": float(vs[0])}
            if vs[0] < best:
                best = vs[0]; torch.save(checkpoint, out / "best.pt")
            if epoch == a.epochs:
                torch.save(checkpoint, out / "last.pt")
            if epoch == 1 or epoch % 5 == 0:
                with torch.no_grad():
                    save_image(model.decode(fixed_z).sigmoid().cpu(), out / f"samples_epoch_{epoch:03d}.png", nrow
=8)
    (out / "timing.json").write_text(json.dumps({"training_seconds": time.perf_counter()-started}))

```

```

if __name__ == "__main__":
    main()
```

## chest_vae/evaluate.py

```
"""Ignite FID/IS, adapted from the assignment's GAN evaluation tutorial.

Uses explicit 2048-dimensional torchvision pool features for the main FID.
Also reports the tutorial-default 1000-probability representation for comparison.
Neither number should be mixed with TensorFlow/pytorch-fid benchmark scores.
"""
import argparse
import hashlib
import json
from pathlib import Path
import time

import numpy as np
import torch
from ignite.engine import Engine
from ignite.metrics import FID, InceptionScore
from ignite.metrics.gan.utils import InceptionModel
from PIL import Image
from torchvision.transforms import functional as TF
from torchvision.utils import save_image

from .data import load_split
from .model import VAE, loss_terms
from .train import device_for


def inception_input(batch):
    # PIL resizing follows the supplied tutorial. Identical processing for both sets.
    images = []
    for x in batch.cpu():
        im = TF.to_pil_image(x.clamp(0, 1)).convert("RGB")
        im = im.resize((299, 299), Image.Resampling.BILINEAR)
        images.append(TF.normalize(TF.to_tensor(im), [0.485, 0.456, 0.406], [0.229, 0.224, 0.225]))
    return torch.stack(images)


def compute_metrics(fake_features, real_features, fake_probs, real_probs):
    # Identity feature extractors allow one shared Inception forward pass.
    evaluator = Engine(lambda engine, batch: batch)
    FID(num_features=2048, output_transform=lambda o: (o[0], o[1])).attach(evaluator, "fid_pool2048")
    FID(num_features=1000, output_transform=lambda o: (o[2], o[3])).attach(evaluator, "fid_probability1000")
    InceptionScore(num_features=1000, output_transform=lambda o: o[2]).attach(evaluator, "inception_score")
    batches = list(zip(fake_features.split(32), real_features.split(32), fake_probs.split(32), real_probs.split(32))
    )
    result = dict(evaluator.run(batches).metrics)
    scores = []
    for probs in torch.tensor_split(fake_probs, 10):
        metric = InceptionScore(num_features=1000)
        metric.update(probs)
        scores.append(metric.compute())
    result.update(is_10split_mean=float(np.mean(scores)), is_10split_std=float(np.std(scores)),
                  is_10split_values=scores)
    return result


def main():
    p = argparse.ArgumentParser()
    p.add_argument("--data", default="data/prepared")
    p.add_argument("--run", default="runs/vae")
    p.add_argument("--batch-size", type=int, default=16)
    p.add_argument("--seed", type=int, default=123)
    p.add_argument("--device", default="auto")
```

```
    a = p.parse_args()
    torch.set_num_threads(6)
    out = Path(a.run); device = device_for(a.device)
    ckpt = torch.load(out / "best.pt", map_location="cpu", weights_only=True)
    config = ckpt["config"]
    model = VAE(config["latent_dim"], config["base_channels"]).to(device)
    model.load_state_dict(ckpt["model"]); model.eval()
    real = load_split(a.data, "test")
    rng = torch.Generator().manual_seed(a.seed)
    z = torch.randn(len(real), config["latent_dim"], generator=rng)
    t = time.perf_counter()
    with torch.inference_mode():
        fake = torch.cat([model.decode(b.to(device)).sigmoid().cpu() for b in z.split(a.batch_size)])
        save_image(fake[:32], out / "generated.png", nrow=8, padding=2)
        mu, _ = model.encode(real[:8].to(device))
        recon = model.decode(mu).sigmoid().cpu()
        save_image(torch.cat([real[:8], recon]), out / "reconstructions.png", nrow=8, padding=2)
        terms = np.zeros(3)
        eps_rng = torch.Generator().manual_seed(a.seed+1)
        for x in real.split(a.batch_size):
            x = x.to(device)
            eps = torch.randn(len(x), config["latent_dim"], generator=eps_rng).to(device)
            logits, mu, logvar = model(x, eps)
            terms += np.array([v.item() for v in loss_terms(logits, x, mu, logvar)])*len(x)
        terms /= len(real)
    np.savez_compressed(out / "generated_samples.npz", images=fake.numpy(), z=z.numpy())
    print(f"Evaluating {len(real)} real and {len(fake)} generated images on {device}", flush=True)
    net = InceptionModel(return_features=False, device=str(device))
    pooled = {}
    hook = net.model.avgpool.register_forward_hook(lambda module, inputs, output: pooled.update(value=output.flatten
     (1)))
    all_features, all_probs = [], []
    with torch.inference_mode():
        for name, images in [("real", real), ("generated", fake)]:
            fs, ps = [], []
            for i, batch in enumerate(images.split(a.batch_size)):
                probs = net(inception_input(batch).to(device))
                fs.append(pooled["value"].cpu()); ps.append(probs.cpu())
                if i % 10 == 0:
                    print(f"{name}: {min((i+1)*a.batch_size,len(images))}/{len(images)}", flush=True)
            all_features.append(torch.cat(fs)); all_probs.append(torch.cat(ps))
    hook.remove()
    # Save intermediate tensors so expensive feature extraction need not be repeated.
    torch.save(dict(real_features=all_features[0], fake_features=all_features[1],
                    real_probs=all_probs[0], fake_probs=all_probs[1]), out / "inception_features.pt")
    result = compute_metrics(all_features[1], all_features[0], all_probs[1], all_probs[0])
    result.update(n_real=len(real), n_generated=len(fake), seed=a.seed, checkpoint_epoch=ckpt["epoch"],
                  checkpoint_sha256=hashlib.sha256((out / "best.pt").read_bytes()).hexdigest(),
                  test_elbo_loss=float(terms[0]), test_bce=float(terms[1]), test_kl=float(terms[2]),
                  feature_model="torchvision Inception_V3_Weights.IMAGENET1K_V1",
                  preprocessing="64x64 grayscale -> uint8 RGB -> PIL bilinear 299x299 -> ImageNet mean/std",
                  fid_note="Ignite FID with 2048 pool features; not TensorFlow/pytorch-fid weights",
                  small_sample_note="Test count is below feature dimension; covariance is rank deficient and FID is
     sample-size biased",
                  seconds=time.perf_counter()-t)
    (out / "metrics.json").write_text(json.dumps(result, indent=2))
    print(json.dumps(result, indent=2), flush=True)


if __name__ == "__main__":
    main()
```

## chest_vae/sample.py

```python
"""Generate new prior samples from a saved VAE, without the training dataset."""
import argparse
from pathlib import Path
import torch
from torchvision.utils import save_image
from .model import VAE
from .train import device_for


def main():
    p = argparse.ArgumentParser()
    p.add_argument("--checkpoint", default="runs/vae/best.pt")
    p.add_argument("--output", default="samples.png")
    p.add_argument("--count", type=int, default=32)
    p.add_argument("--seed", type=int, default=2026)
    p.add_argument("--device", default="auto")
    a = p.parse_args()
    if a.count < 1:
        p.error("--count must be positive")
    ckpt = torch.load(a.checkpoint, map_location="cpu", weights_only=True)
    c = ckpt["config"]; device = device_for(a.device)
    model = VAE(c["latent_dim"], c["base_channels"]).to(device)
    model.load_state_dict(ckpt["model"]); model.eval()
    z = torch.randn(a.count, c["latent_dim"], generator=torch.Generator().manual_seed(a.seed))
    with torch.inference_mode():
        images = torch.cat([model.decode(b.to(device)).sigmoid().cpu() for b in z.split(64)])
    Path(a.output).parent.mkdir(parents=True, exist_ok=True)
    save_image(images, a.output, nrow=8, padding=2)


if __name__ == "__main__":
    main()
```

## scripts/download_data.sh

```bash
#!/usr/bin/env bash
set -euo pipefail
cd "$(dirname "$0")/.."
mkdir -p data
curl -fL -C - --retry 5 --retry-all-errors \
  'https://www.kaggle.com/api/v1/datasets/download/paultimothymooney/chest-xray-pneumonia' \
  -o data/chest-xray-pneumonia.zip
python -m chest_vae.data
```

## scripts/recover_dataset.py

```python
"""Recover the complete nested dataset copy from an interrupted Kaggle ZIP.

The Kaggle archive stores duplicate dataset trees. This optional recovery tool
reads local ZIP/ZIP64 headers, checks decompressed sizes and CRC32 values, and
requires the known full image counts before accepting a recovered archive.
Normal downloads should use download_data.sh instead.
"""
from collections import Counter
from pathlib import Path
import struct
import zipfile
import zlib

source = Path("data/chest-xray-pneumonia.zip")
target = Path("data/chest-xray-recovered.zip")
prefix = "chest_xray/chest_xray/"
expected = {"train": 5216, "val": 16, "test": 624}
counts = Counter()
with source.open("rb") as f, zipfile.ZipFile(target, "w", compression=zipfile.ZIP_STORED) as out:
```

```
    while True:
        h = f.read(30)
        if len(h) != 30 or h[:4] != b"PK\x03\x04":
            break
        v = struct.unpack("<4s5H3I2H", h)
        _, _, flags, method, _, _, crc, compressed, size, namelen, extralen = v
        name = f.read(namelen).decode("utf-8")
        extra = f.read(extralen)
        if flags & 8:
            raise ValueError("Streaming data descriptors are not supported")
        pos = 0
        while pos+4 <= len(extra):
            tag, n = struct.unpack_from("<HH", extra, pos)
            chunk = extra[pos+4:pos+4+n]; pos += 4+n
            if tag == 1:
                j = 0
                if size == 0xffffffff:
                    size = struct.unpack_from("<Q", chunk, j)[0]; j += 8
                if compressed == 0xffffffff:
                    compressed = struct.unpack_from("<Q", chunk, j)[0]
        if f.tell()+compressed > source.stat().st_size:
            break
        wanted = name.startswith(prefix) and name.lower().endswith(".jpeg")
        if not wanted:
            f.seek(compressed, 1)
            continue
        raw = f.read(compressed)
        payload = zlib.decompress(raw, -15) if method == 8 else raw if method == 0 else None
        if payload is None or len(payload) != size or zlib.crc32(payload) != crc:
            raise ValueError(f"Corrupt archive member: {name}")
        relative = name[len(prefix):]
        counts[relative.split("/")[0]] += 1
        out.writestr("chest_xray/"+relative, payload)
if dict(counts) != expected:
    target.unlink()
    raise ValueError(f"Incomplete dataset copy: {counts}, expected {expected}")
print(f"CRC-verified complete dataset: {dict(counts)} -> {target}")
```

## tests/test_vae.py

```
import torch
from chest_vae.model import VAE, loss_terms
from chest_vae.data import patient_key


def test_loss_reduction_and_standard_normal_kl():
    x = torch.zeros(2, 1, 64, 64)
    mu = torch.zeros(2, 8)
    loss, bce, kl = loss_terms(x, x, mu, mu)
    assert torch.allclose(bce, 4096 * torch.log(torch.tensor(2.)))
    assert kl.item() == 0
    assert loss == bce


def test_sampling_shape_and_backward():
    torch.manual_seed(3)
    model = VAE(8, 8)
    x = torch.rand(2, 1, 64, 64)
    logits, mu, logvar = model(x)
    assert logits.shape == x.shape
    assert mu.shape == logvar.shape == (2, 8)
    loss, _, _ = loss_terms(logits, x, mu, logvar)
    loss.backward()
    assert all(p.grad is not None and torch.isfinite(p.grad).all() for p in model.parameters())
    assert model.mu.weight.grad.abs().sum() > 0
    assert model.logvar.weight.grad.abs().sum() > 0
    assert model.decode(torch.randn(3, 8)).shape == (3, 1, 64, 64)

```

```

def test_patient_grouping():
    assert patient_key("person123_bacteria_4.jpeg") == patient_key("person123_virus_5.jpeg")
    assert patient_key("NORMAL2-IM-1427-0001.jpeg") == "NORMAL2-IM-1427"
    assert patient_key("IM-0115-0001.jpeg") == "IM-0115"
```

## tests/test_metrics.py

```python
import torch
from ignite.metrics import FID, InceptionScore


def test_identical_features_have_zero_fid():
    x = torch.tensor([[0., 0.], [1., 0.], [0., 1.], [1., 1.]])
    metric = FID(num_features=2)
    metric.update((x, x))
    assert abs(metric.compute()) < 1e-7


def test_constant_probabilities_have_is_one():
    metric = InceptionScore(num_features=3)
    metric.update(torch.tensor([[0.2, 0.3, 0.5]]).repeat(10, 1))
    assert abs(metric.compute() - 1.) < 1e-6
```